# Short presenter script + Q&A

*Six slides • approximately 5–6 minutes including the live code run*

## The simple plan

Explain the joint and two training choices. Run the program. Show the terminal results and open its graphs. Interpret the measured difference, then connect the idea to the readings and a quadruped joint.

### 1. Run your working code

.\.venv\Scripts\python.exe reading5_sim2real_ppo.py

Use the project folder and Python 3.12 environment that already work in VS Code. Briefly point to Joint.step(), train(), and main(). The program trains both PPO agents, evaluates both conditions, prints statistics, and saves three PNGs. The verified cloud run took 27.72 seconds; local timing varies. Rerunning this command replaces its generated results and models.

### 2. Show the results

Point out the fixed/random policy rows and fixed/real environment labels in the terminal. Here, real means held-out simulation. Open fig1_training_rewards.png briefly: recorded rewards show learning. Use slides 3–4 for readable tracking and success graphs, or open fig2_real_rollouts.png and fig3_success_rates.png directly.

Reported held-out results: fixed 14/30 successes and 0.3299 rad mean RMSE; randomized 29/30 and 0.2250 rad. Both achieve 30/30 nominal success. Success requires a full five-second episode with RMSE < 0.32 rad and no safety termination.

### Optional: repeat quickly with saved models

.\.venv\Scripts\python.exe classroom\demo.py --trials 30 --rounds 1 --open-figure

Or double-click classroom\run_demo.cmd. This replays the same full evaluation without retraining, writes a new timestamped folder, and preserves the original outputs. Say clearly that this is saved-policy evaluation. If a live run fails, show the saved figures and identify them as previously executed results.

# Speaking script: slides 1–3

## 1. What am I demonstrating?

SAY: My project tests whether varied training physics improves a learned controller's robustness. I model one robotic joint, where a PPO policy chooses a target angle and a PD controller supplies the torque. One policy trains with fixed physics; the other experiences randomized inertia, friction, and delay. I compare their joint tracking in nominal and unfamiliar simulations.

DO: Point to the single joint, the fixed-physics policy, and the randomized-physics policy.

NEXT: I will run the program and briefly show where that comparison happens.

## 2. Run the code and follow the workflow

SAY: I run reading5_sim2real_ppo.py using the project's Python environment. In Joint.step, the policy's angle target becomes a saturated PD torque, then the joint dynamics advance. In train, Stable-Baselines3 PPO learns from tracking rewards. The main function checks the environment, trains both policies, evaluates thirty matched episodes per policy and condition, prints the metrics, and creates three graphs. The saved run took about twenty-eight seconds on the cloud CPU.

DO: Run .\.venv\Scripts\python.exe reading5_sim2real_ppo.py. Show Joint.step and train while it runs, then point to the terminal results and generated PNGs.

NEXT: The graphs make the learning and tracking behavior easier to interpret.

## 3. Does the joint follow the target?

SAY: The terminal shows evaluation statistics, and the program also saves the training-reward graph. Here I focus on the held-out tracking plot. The dashed line is the desired angle; the colored line is the joint's actual angle. The randomized policy tracks more smoothly in this example. Both policies pass this seed, so overall success needs all thirty trials.

DO: Briefly open fig1_training_rewards.png during the live run, then show the full tracking plot. Identify desired versus actual angle; seed 1000 is one preselected example.

NEXT: The success-rate graph summarizes all thirty evaluation trials per condition.

# Speaking script: slides 4–6

## 4. What do the results mean?

SAY: Both policies succeeded in all thirty nominal trials. In the unfamiliar simulation, the fixed policy succeeded in fourteen of thirty trials, compared with twenty-nine of thirty for the randomized policy. Mean tracking RMSE decreased from 0.3299 to 0.2250 radians. This supports better robustness for this trained policy pair, while one training seed and simplified physics limit the conclusion.

DO: Point to 14/30 versus 29/30, the held-out RMSE values, and the nominal 100% bars. Success requires RMSE below 0.32 rad for a full safe episode.

NEXT: That result illustrates the sim-to-real ideas discussed in the assigned readings.

## 5. How does this connect to Reading 5?

SAY: Tan and colleagues motivate randomized simulation and careful actuator modeling for robot transfer. Zhao and colleagues place these methods within the broader reality-gap problem. Sutton and Barto explain policy-gradient foundations; PPO is the specific clipped-surrogate actor–critic method used here. MuJoCo represents richer multibody and contact simulation. My simple joint illustrates these ideas without reproducing full robot physics.

DO: Point to each source and its one-line connection. MuJoCo is a reference connection, not a dependency used by this program.

NEXT: A practical application is a robot joint whose physical behavior changes during use.

## 6. A real robotics application and the takeaway

SAY: In a quadruped inspired by Go2 or ANYmal, payload changes can alter effective joint inertia, wear can change friction, and communication can introduce delay. Randomized training is one way to prepare a controller for that variation. My result demonstrates this idea in simulation. It does not establish physical robot transfer, balance, or walking; those require richer models and hardware validation.

DO: Connect inertia to load, friction to wear, and latency to actuator communication. End with the measured held-out improvement and its simulation-only scope.

NEXT: I can rerun the code or return to a graph for questions.

## Reading links

Tan et al. (2018): https://arxiv.org/abs/1804.10332

Zhao et al. survey: https://arxiv.org/abs/2009.13303

Sutton & Barto, Ch. 13: http://incompleteideas.net/book/the-book-2nd.html

MuJoCo: https://mujoco.org/

Specific PPO method: https://arxiv.org/abs/1707.06347

# Quick Q&A

## 1. Is this an actual physical sim-to-real experiment?

The held-out environment is a simulated approximation of unfamiliar reality. It uses greater inertia, friction, delay, and sensor noise, but no calibrated robot or hardware trial. Physical transfer would require a realistic model, reliable sensing, protective limits, and supervised experiments.

## 2. Why would domain randomization help?

Randomization reduces dependence on one exact training model by varying inertia, friction, and delay at each reset. It may encourage a controller that tolerates uncertainty. It is not guaranteed to help: the training range, learning budget, and held-out condition all affect the outcome.

## 3. Why does PPO choose an angle instead of torque?

PPO supplies a bounded target angle, while PD feedback converts the difference between target and measured state into torque. This separates the learned decision from a familiar actuator-control structure. Torque saturation is explicit, but this architecture alone does not prove closed-loop safety or stability.

## 4. How does PPO differ from general policy-gradient methods?

Policy-gradient methods adjust policy parameters to improve expected return. PPO is a specific method that uses a clipped surrogate objective to discourage overly large policy updates, together with a learned value function. Sutton and Barto provide the broader foundations rather than this particular implementation.

## 5. What do these results actually establish?

The randomized policy achieved 29/30 held-out successes versus 14/30 for the fixed policy, with lower mean RMSE. Both had 30/30 nominal success. Only one training seed per policy was used, so this is a measured case study rather than a general superiority or hardware-transfer claim.

## 6. How can I repeat the classroom demonstration?

The main program can be rerun and regenerates its outputs. For evaluation without retraining or replacing those files, use classroom/demo.py --trials 30 --rounds 1. Each helper launch gets a new folder. Reusing seeds checks execution repeatability, not additional independent evidence.